ClarifySQL

Teaching a small language model when to ask before it writes SQL

Final project report — draft

Language Models and Agents

GUILD4AGENTS

Subhash Dangeti (2025201007)

Madha Manish (2025202004)

Mannem Tejaswini (2025201002)

Sathvik Reddy (2025201020)

8 November 2026

github.com/Sathviktr/LMA_ClarifySQL

Status of this draft. The final test experiments, required ablations, CoSQL analysis and scripted GPU demo are complete, and one browser clarification exchange is documented. Independent question-usefulness and simulator reviews and external adapter-access checks remain pending. Browser evidence covers one interaction. This report includes those limits; the earlier mid-submission report is kept unchanged.

1 What we studied

If someone asks, "Which is the best department?", the database cannot tell us what the person means by "best". It could mean average marks or placement rate. A model may produce valid SQL while silently choosing the wrong meaning.

We built an agent around Qwen3-4B-Instruct-2507 [5]. It can inspect the database, ask a question, write SQL when enough information is available, or explain why a request cannot be answered. We train a small adapter for choosing these actions. The SQL writer stays fixed so that changing the writer does not explain a difference between the clarification policies.

Our two main questions are:

1. Does a trained policy work better than a prompted policy with the same base model and tools?
2. Does writing explicit missing-information slots help compared with a policy that omits slots?

A slot is a missing detail such as an age cutoff or ranking measure. Our final test supports the first comparison on execution and decision F1. It does not establish an execution advantage for explicit slots. The trained policies also abstain less reliably on requests for missing data.

2 System design

The policy chooses one JSON action at a time. The controller validates the action, stores replies and tool results, and enforces the episode limits.

Action	What happens next
Inspect	Describe a table, inspect example values, or search a stored glossary definition.
Clarify	Ask a natural-language question and add the reply to conversation memory.
Execute	Give the rewritten request to the frozen SQL writer and execute the SQL read-only.
Abstain	Name the required information that is missing from the database and stop.

Request, compact schema, tool results and memory ⟶ policy adapter.
Inspect ⟶ tool result ⟶ policy.
Clarify ⟶ reply and memory ⟶ policy.
Execute ⟶ frozen writer ⟶ read-only SQL result.
Abstain ⟶ missing-information explanation.

The main policy sees a compact schema catalog and can request more information with tools. The writer gets the database schema and the rewritten request. The adapter is enabled for the trained policy and disabled for both the writer and the simulated user. The benchmark simulated user reads the natural-language question and hidden intended details, chooses the relevant details, and returns prepared answers. It does not use the policy's slot fields, which keeps that part of the comparison the same for both policies.

Each episode allows at most three inspections, two clarification questions, two SQL repairs and eight policy/writer model calls. Simulator calls are outside that limit but are included in total cost reporting. SQL errors and omitted numeric values from actual replies can trigger repair. Checking that a number appears in SQL does not establish that its column, comparison or meaning is correct.

SQL execution uses a fixed database path, a read-only connection, a single-SELECT/WITH check and an SQLite authorizer that blocks writes and other disallowed operations. Row and time limits constrain query execution. The fictional-data demo also removes sample-value inspection from its allowed tools.

3 Data and training

We use Spider database questions and SQL [1] to construct clear and vague requests, CoSQL dialogue turns [2] for a separate natural-conversation check, and TrustSQL examples [3] for requests that cannot be answered from the available information. The episode splits are organized by database.

Split	Episodes	Policy training/evaluation states
Training	307	602
Validation	53	101
Final test	188	Not used for training or checkpoint choice

The test set covers 20 databases and contains 79 clear, 47 single-slot ambiguous, 8 two-slot, 14 definition-resolved and 40 unanswerable episodes. Thus execution accuracy has 148 answerable cases. The separate reviewed CoSQL slice has 59 turns: 45 execute, 13 clarify and 1 abstain. Earlier dialogue is included in the CoSQL input; only the first non-inspect decision is scored.

We train the policy with QLoRA [4]: the base model is frozen in 4-bit NF4 storage, and low-rank adapter weights are trained. The saved recipe uses rank 16, alpha 32, dropout 0.05, a micro-batch of one state and gradient accumulation of 16. The main run has 33,030,144 trainable parameters, a maximum sequence length of 1,024, three epochs, a learning rate of 2×10^{-4} and 6% warmup. Loss is computed on target action tokens, with prompt tokens masked. We trained main seeds 1 and 2 and a separate no-slot adapter. Final inference uses FP16 weights and greedy decoding on free Kaggle T4s.

Checkpoint selection used validation, not the final test. The recorded rule compares validation decision F1 rounded to 0.01, then slot accuracy, then the recorded candidate order. Three candidates round to 0.98; two also tie on slot accuracy. The selected recipe is learning rate 2×10^{-4}, epoch 3. This is a selection rule, not evidence that the tied scores are statistically equivalent.

Before the final runs, eight test reference phrases were corrected for scoring in `data/episodes/slot_terms.json`. The episode requests, training states, hidden simulator details and model outputs were not rewritten. The automatic slot score remains a lexical proxy.

4 Experiments and scoring

Condition	Purpose
B1	Direct SQL generation without clarification or repair.
B2	Direct SQL generation with up to two repairs.
B3	Prompted agent with worked examples, the same base model, tools, writer and limits.
SFT seeds 1 and 2	Main trained policy with explicit slots; one run per training seed.
A2	Separately trained no-slot policy, keeping the other main settings.
A3	Seed-1 adapter with full schema and all glossary definitions in the policy prompt.
Oracle	Fully specified source request given to the fixed writer without repairs.

The oracle is a reference run, not a mathematical upper bound. Its writer can still make mistakes, and other conditions have repair opportunities. B1, B2 and the oracle always use an execute decision, so their decision F1 is identical under the fixed test labels.

We report execution accuracy on the 148 answerable cases and decision macro-F1 over all 188 episodes. Decision F1 treats clarify, execute and abstain equally by averaging their class F1 values. We also use the official test-suite evaluator [6], with the predicted values preserved, to check SQL on database variants. Correct abstention is measured on 40 unanswerable cases and requires an abstention that names the missing information under an automatic word-overlap check. Unnecessary-question rate is measured on the 79 clear requests.

For uncertainty, we resample all 20 databases with replacement 5,000 times. Both policies are evaluated on the same resampled databases. For each draw we keep both seed comparisons and their mean. Resampling databases preserves related questions within each database; it does not measure variation over all possible training seeds or eliminate simulator error. The first comparison below is primary; ablations and the CoSQL slice are exploratory.

5 Final test results

System	Correct/148	Exec. (%)	Test-suite (%)	Decision F1	Abstain/40
B1	84	56.76	46.62	0.2207	0
B2	89	60.14	47.30	0.2207	0
B3	94	63.51	54.05	0.7850	35
SFT seed 1	114	77.03	67.57	0.8790	29
SFT seed 2	112	75.68	65.54	0.9257	29
A2: no slots	115	77.70	68.24	0.8923	28
A3: full context	108	72.97	62.16	0.8533	28
Oracle	116	78.38	68.92	0.2207	0

The trained seeds average 76.35% execution accuracy, compared with 63.51% for B3. The mean gain is **12.84 percentage points**, with a 95% paired database interval of **[5.55, 19.08]**. Decision F1 improves by 0.11735 on average, with interval [0.06545, 0.17945]. Both primary intervals exclude zero under the resampling assumptions.

Comparison	Execution gain (points)	95% paired interval
Seed 1 minus B3	13.51	[5.48, 20.63]
Seed 2 minus B3	12.16	[5.47, 17.90]
Two-seed mean minus B3	12.84	[5.55, 19.08]

The gain does not mean every behavior improved. Both trained seeds correctly abstain on only 29 of 40 unanswerable cases, compared with 35 for B3. This is 72.5% versus 87.5% under the automatic check. Looking only at the final action, B3 abstains on 39 and executes on 1 of these requests. Seed 1 abstains on 32 and executes on 8; seed 2 abstains on 33 and executes on 7. Thus the difference includes more executions on unanswerable requests, as well as explanations that miss the naming check. Also, lower test-suite scores than original-database execution scores show that a matching answer on one database is not enough evidence about SQL meaning.

5.1 Where the gain appears

Request type	Cases	B3 correct	Seed 1 correct	Seed 2 correct
Clear	79	56	67	68
Single-slot ambiguous	47	31	35	34
Two-slot	8	4	4	3
Definition-resolved	14	3	8	7

B3 asks questions on 21 of 79 clear requests; seed 1 does so on 2 and seed 2 on none. The gains include avoiding unnecessary questions and handling definitions better. Two-slot requests remain difficult. These automatic results do not by themselves establish that the trained policy asks more useful questions.

5.2 Ablations

A2 solves 115 answerable cases, one more than seed 1. Its paired execution difference interval includes zero (about -1.42 to $+3.20$ points), as does its decision-F1 interval. One case is about 0.68 percentage points on this test. We therefore do not claim an execution benefit from explicit slots. Their effect on human-rated question usefulness remains unresolved.

A3 has lower point estimates than seed 1. Its execution-difference interval touches zero and its decision-F1 interval includes zero. Adding all context gives no observed improvement in this run. That does not prove that retrieval is necessary for every database or that full context can never help.

6 Natural dialogue and computation

CoSQL condition	Correct/59	Decision F1	Clarify recall/13
B3	26	0.3740	8
SFT seed 1	40	0.4560	3
Always execute (label-only)	45	0.2883	0

SFT improves overall decision accuracy over B3, but it detects fewer genuine clarification turns. An always-execute policy has higher accuracy because 45 labels are execute, although its macro-F1 is lower. This small, imbalanced slice provides mixed evidence about natural-dialogue generalization. The single abstention example cannot establish reliable abstention behavior.

Test condition	Total calls	Total input tokens	Generation seconds
B3	862	605,038	1,198.2
SFT seed 1	570	216,401	528.5
SFT seed 2	552	207,348	558.2
A2	573	205,609	479.5
A3	525	273,484	664.9

These totals include policy, writer and simulator inference. Generation time is elapsed time in generation calls, not isolated GPU-kernel time. The two-GPU inference cell took about 40.6 minutes, excluding later CPU scoring. B3 reaches an episode limit twice; the main trained seeds have no limit outcomes. Seed 2 still has a nonzero invalid output-attempt rate (0.006), so recovered retries should not be described as every generated output being valid. No paid model API was used for these experiments.

7 Fictional university demo

The demo uses made-up departments, students, marks and placement flags. The same session engine can pause for a real reply in Gradio or receive a predefined reply during an automatic check. Model text and SQL appear in plain text boxes. Queries use the same read-only executor, and sharing requires credentials.

Notebook 08 loads seed 1 once and runs seven fixed examples. It saves every outcome, including failures, to `checkpoint8_results.zip`. Reference SQL is used after execution for scoring, never in the policy or writer prompt. Its original saved result is **5/7**, reproduced locally from the received traces.

Example	Measured behavior
Department names	Correctly returns all three departments.
High marks	Retrieves the above-80 definition and returns the correct students.
Best by average marks	Returns Mathematics after the fixed preference reply.
Best by placement rate	Returns Computing on the seed data, but the query has a counting error.
Older with strong marks	Both intended-age variants fail: the agent never asks and guesses age above 18.
Student emails	Correctly names the missing email information and abstains.

7.1 A hidden false positive

The placement query divides `COUNT(p.student_id)` by `COUNT(s.id)`. Every student has a placement row, including students with `placed=0`. All departments therefore receive predicted rate 1, and Computing happens to be returned after the tie. The intended rate uses the indicator, such as `1.0*SUM(p.placed)/COUNT(*)`.

After inspecting that query, we created two additional fictional-data variants. In one, only the two Mathematics students are placed; in the other, only the two Physics students are placed. The original predicted SQL still returns Computing in both. These CPU counterexamples show that the query does not represent the intended ranking. They are post-hoc checks, not a replacement for the original 5/7 or a new benchmark score.

Both ranking questions also offer "age or id", although the department table has no age column. The fixed reply supplies a criterion outside those choices. Thus a successful final result does not establish useful question wording. In the two-slot cases, the glossary matches strong marks to high marks, but

the age condition remains unresolved. The numeric reply check cannot help when the policy never asks for a value.

Gradio construction succeeds with version 6.30.0. The archive does not demonstrate browser clicks, sharing or an actual user conversation. The scripted model run uses 21 calls and about 56.4 seconds of generation time; model loading takes about 48.8 seconds separately.

7.2 One documented browser exchange

A two-page browser printout supplied on 8 November shows the request "Which is the best department?", the question "Ranked by what: age or id?", the reply "age", a completed-query status and Physics in the result table. The shown SQL joins departments to students and orders by `MAX(s.age)`. It therefore selects the department containing the oldest student. A CPU replay on the saved fictional database also returns Physics, whose oldest student is 25.

The capture documents a displayed request/reply/result path. It does not show the login sequence or record the browser session's checkpoint and database hashes. Also, "age" leaves maximum versus average age unspecified; both happen to select Physics on this fixture. The capture supports interface use, not a new benchmark score or a claim that every ambiguity was resolved. Its observations and file hash are recorded separately from the scripted run.

8 Limitations and work still open

- **Human question evaluation is incomplete.** The planned sheets cover the same 40 ambiguous episodes for B3, seed 1 and A2, giving 120 items per rater. Two independent ratings have not been collected, so we report no human usefulness score or agreement statistic for this study. Earlier dataset annotation does not replace this evaluation.
- **Simulator reliability is not independently established.** The review sheets contain 247 actual decisions: 50 each for B3, both main seeds and A2, and all 47 available for A3. The shortfall is disclosed rather than filled with duplicates. These human judgments remain pending.
- **The data and statistics are limited.** The benchmark includes constructed vague requests, only eight two-slot test episodes and two training seeds. The database bootstrap does not cover every source of uncertainty. The CoSQL slice is small and dominated by execute labels.
- **Automatic checks have gaps.** Lexical slot and missing-information checks do not measure semantic correctness. A numeric value in SQL may be used with the wrong column or comparison. The demo shows both a guessed cutoff and SQL that succeeds by coincidence on seed data.
- **Browser coverage is limited.** One request/reply/result path is documented, while the login sequence, other examples and error paths were not captured. Software fixtures test repair and blocked writes, but they should not be described as a completed model-driven safety study.

The remaining submission work is to complete the independent reviews, extend browser coverage, confirm instructor access to the saved weights and finalize the artifact package. Improving question grounding, checking unresolved intent before execution and testing SQL on more variants are useful follow-ups. Any new intervention should be reported as a new version rather than silently changing the already measured benchmark conditions.

9 Reproduction and artifact status

The code, derived episodes, training logs, final traces and analysis are in https://github.com/Sathviktr/LMA_ClarifySQL. Raw benchmark databases and large weights are kept outside Git. The saved phase6 notebook output, run 363373340, contains `sft_s1`, `sft_s2` and `a2_s1`; instructor access

to that output has not been verified. The selected seed-1 weight hash agrees between the benchmark and the demo.

Artifact	Repository location or recorded source
Final test and CoSQL	`results/final/`, source commit `428e743`
Paired intervals	`results/final/analysis/paired_bootstrap.json`
Original demo archive	`results/demo/checkpoint8/received.zip`, source `0d75a73`
Demo verification and variants	`results/demo/checkpoint8/analysis/`
Browser observation	`results/demo/browser_evidence_2026-11-08.json`
Training and selected recipe	`results/train/` and saved phase6 output
GPU workflows	Notebooks 01–08 and their readable builder scripts

To reproduce CPU verification after installing the project and setting `CLARIFYSQL_DATA` to the downloaded benchmark directory:

```
python scripts/verify_final.py
python scripts/analyze_demo.py
```

Both verification commands and the 40 selected tests also pass from a new local clone of commit `34500af`, using the existing venv and external benchmark data. This is a clean-code check, not a fresh dependency installation or another GPU run. Test-suite means are recomputed from saved flags; the official variant queries are not repeated in that CPU check. The exact scope and commands are saved in `results/reproducibility_checkpoint8.json`.

Frozen-run manifests check code, data and weight fingerprints before resuming. Pin the recorded source revision to resume an original run; later source changes must not be mixed into those traces. The original demo ZIP is preserved byte-for-byte and includes the tiny fictional database. Source verification reads Git blobs so Windows newline conversion does not create a false mismatch.

A targeted local check passes 45 tests covering the demo, credential-relaunch workflow, received-output analysis, notebook consistency and final-evaluation code. These are software tests, not additional model results. An earlier combined Windows CPU run has a native PyTorch failure in the tiny training fixture; that fixture passes separately, and the actual GPU training runs completed. We do not claim that every test passed together in one Windows process.

10 What we conclude

Within this fixed experiment, training improves execution and decision F1 compared with prompting. The gains include fewer unnecessary questions and better use of definitions, while abstention becomes less reliable. We do not establish an execution advantage from explicit slots or strong generalization to real clarification turns. The demo makes the remaining problems concrete: poorly grounded choices, unresolved thresholds and an incorrect query that happens to return the expected answer. These limits are part of the result and guide the next version of the system.

References

[1] T. Yu et al. *Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task*. 2018. https://arxiv.org/abs/1809.08887.

[2] T. Yu et al. *CoSQL: A Conversational Text-to-SQL Challenge Towards Cross-Domain Natural Language Interfaces to Databases*. 2019. https://arxiv.org/abs/1909.05378.

[3] TrustSQL project. Dataset and implementation. https://github.com/glee4810/TrustSQL.

[4] T. Dettmers et al. *QLoRA: Efficient Finetuning of Quantized LLMs*. 2023. https://arxiv.org/abs/2305.14314.

[5] Qwen team. *Qwen3-4B-Instruct-2507 model card*. https://huggingface.co/Qwen/Qwen3-4B-Instruct-2507.

[6] Official text-to-SQL test-suite evaluator. https://github.com/taoyds/test-suite-sql-eval.